Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Laboratory Work 3:

Track A: The Command Line (Shell, Pipes and Text Filters)

Elaborated: gr. FAF-243
Poiata Calin

Verified:
Patricia Reițman

Chișinău – 2026

Table of Contents

Environment & Toolchain 1
Purpose of the Laboratory Work 1
Setup & Dataset Initialization 1
Part 1. Move Fast, Find Help 2
1. Interactive Tab Completion & History Lookup 2
2. Identifying Built-in vs. External Commands 2
3. Inspecting the `$PATH` Environment Variable 2
4. Exit Code of an Unresolved Command 2
5. Built-in Help and Offline Documentation 3
Observe (Part 1 Answers) 3
Part 2. What the Shell Does to Your Line 3
1. Test Directory Setup 3
2. Prediction vs. Real Output Analysis 3
3. Globbing & Expansion Verification 4
4. Word Splitting on Whitespace and Escaping 4
5. Variable Quoting Semantics 5
Observe (Part 2 Answers) 5
Part 3. Redirection, Chaining and Jobs 5
1. Separating Standard Streams (`stdout` vs `stderr`) 5
2. Comparing Redirection Evaluation Order 5
3. Chaining Operators and Stdin Redirection 6
4. Background Job Control 6
Observe (Part 3 Answers) 6
Part 4. Answer Questions with One-Liners 7
Q1. How many requests failed? 7
Q2. How many different pages were requested, and which? 7
Q3. How many requests did each page get? 7
Q4. Which user or users have the most failed requests, and how many? 8
Q5. How many requests did user3 make, and how many of them failed? 8
Q6. Print the last 3 failed requests, showing only time and user. 8
Q7. Which login shells appear in /etc/passwd, and how many accounts use each? 8
Q8. PREDICT, then explain: why do `ls /etc | wc -l` and `ls -l /etc | wc -l` differ by exactly one? 8
Bonus (+1): Regex for Minute Ending in 0 and Request Failed 9
Full Run of `oneliners.sh` 9
Part 5. Your First Script 9
1. Script Implementation (`report.sh`) 9
2. Execution on Valid Access Log 10
3. Defensive Error Handling on Missing File 10
Observe (Part 5 Answers) 10
Bonus (+1): Scheduled Execution via Cron 11
Conclusions 12

Environment & Toolchain

- **Host OS:** Linux (Arch Linux x86_64, Kernel 7.2.9-arch1-1)
- **Shell:** GNU bash, version 5.3.20(1)-release (`x86_64-pc-linux-gnu`)
- **Workspace Directory:** `/home/calin/code/os-labs/lab3`
- **Core Utilities:** GNU Coreutils (`ls` , `cat` , `wc` , `head` , `tail` , `cut` , `tr` , `sort` , `uniq` , `touch` , `date` , `whoami`)
- **Filter & Text Tools:** GNU `grep` , `man-db` (`man` , `apropos`), Bash built-ins (`type` , `cd` , `echo` , `help` , `jobs` , `kill`)
- **Automation:** Cron (`cronie` / `/usr/bin/crontab`)

Purpose of the Laboratory Work

1. Master interactive command-line navigation, reverse incremental history retrieval (`Ctrl+R`), programmable tab completion, and manual inspection (`man` , `--help` , `help`).
2. Demystify the Unix shell execution cycle by exploring the exact sequence of tokenization, brace expansion, variable/parameter substitution, command substitution, arithmetic expansion, word splitting, pathname expansion (globbing), and quote removal.
3. Master standard I/O streams and file descriptors (`stdin` 0, `stdout` 1, `stderr` 2), file descriptor duplication (`2>&1`), stream redirection (`>` , `>>` , `<` , `2>`), command chaining operators (`;` , `&&` , `||`), and asynchronous background job control (`&` , `jobs` , `kill`).
4. Construct composable, multi-stage Unix filter pipelines combining `grep` , `cut` , `sort` , `uniq` , `wc` , `head` , and `tail` to analyze structured system logs without writing monolithic code.
5. Author a robust, production-style POSIX Bash script (`report.sh`) implementing defensive argument validation, proper stream separation (`>&2`), meaningful exit codes, and automated scheduled execution via `cron` .

Setup & Dataset Initialization

Prior to executing the analytical tasks, an artificial web access log consisting of 500 requests was generated according to the specification. Each log entry comprises five fields: date, timestamp, user identifier (`user0` .. `user6`), request outcome (`OK` or `FAIL`), and requested endpoint (`/page0` .. `/page3`).

```
❱ for i in $(seq 1 500); do
    echo "2026-10-05 10:$(printf %02d $((i % 60))) user$((i % 7)) $([ $((i % 5)) -eq 0 ]
&& echo FAIL || echo OK) /page$((i % 4))"
  done > access.log
❱ head -3 access.log ; wc -l access.log
2026-10-05 10:01 user1 OK /page1
2026-10-05 10:02 user2 OK /page2
2026-10-05 10:03 user3 OK /page3
500 access.log
```

The verification command confirmed exactly 500 lines generated with consistent formatting across all columns.

Part 1. Move Fast, Find Help

1. Interactive Tab Completion & History Lookup

- **Tab completion:** Typing `ls /usr/sh` and pressing `Tab` twice causes the shell readline engine to inspect candidate directory entries matching `/usr/sh*` and auto-complete to `/usr/share/`. Typing `cd /et` and pressing `Tab` once completes immediately to `cd /etc/` because `/etc` is the unique matching directory in the root hierarchy.

```
❱ ls /usr/sh<Tab><Tab>
❱ ls /usr/share/
❱ cd /et<Tab>
❱ cd /etc/
```

Figure 1.1 – Tab completion

- **Reverse Incremental History Search (`Ctrl+R`):** Pressing `Ctrl+R` activates readline reverse incremental search mode. Typing the substring `access` matches the most recent matching command from the in-memory history buffer:

```
(reverse-i-search)`access': head -3 access.log ; wc -l access.log
```

Pressing `Enter` executes the recalled pipeline immediately without needing to re-type it.

2. Identifying Built-in vs. External Commands

The `type` and `which` utilities determine how command names are resolved by the shell:

```
❱ type cd ; type ls ; type echo ; which ls
cd is a shell builtin
ls is /usr/bin/ls
echo is a shell builtin
/usr/bin/ls
```

Figure 1.2 – Output of type and which

3. Inspecting the `$PATH` Environment Variable

The shell searches the directories registered in `$PATH` sequentially from left to right when resolving non-builtin command names without path separators:

```
❱ echo $PATH | tr ':' '\n'
/home/calin/.gemini/antigravity-cli/bin
/home/calin/.local/bin
/home/calin/.local/bin
/usr/local/bin
/usr/bin
/usr/local/sbin
/opt/android-sdk/platform-tools
/var/lib/flatpak/exports/bin
/usr/lib/jvm/default/bin
/usr/bin/site_perl
/usr/bin/vendor_perl
/usr/bin/core_perl
```

Figure 1.3 – The directories in PATH

4. Exit Code of an Unresolved Command

When an executable cannot be found in any directory listed in `$PATH`:

```
❱ nosuchprog ; echo "exit code: $?"
bash: line 1: nosuchprog: command not found
exit code: 127
```

Figure 1.4 – Exit code of a missing command

5. Built-in Help and Offline Documentation

- `man ls` : Invokes the man-db pager (`less`) for the external `ls` utility. Inside the manual, typing `/sort` searches forward, `n` jumps to the next occurrence, and `q` terminates the pager.
- `ls --help | head` : Displays standard command-line flags and syntax documentation directly from the `ls` binary.
- `man -k "list directory"` : Queries the system-wide apropos database for manual entries matching directory listing keywords.
- `help cd` : Displays usage documentation for Bash built-in commands which do not provide standalone man pages.

Observe (Part 1 Answers)

1. **Which of `cd` , `ls` , `echo` are built-ins, and why MUST `cd` be one?**
   - `cd` and `echo` are shell built-ins. `ls` is an external binary located at `/usr/bin/ls` .
   - `cd` **must** be a built-in because in Unix-like operating systems, child processes cannot alter the execution environment of their parent process. When an external program is invoked, the shell creates an isolated child process via the `fork()` and `execve()` system calls. If `cd` were an external binary, it would execute `chdir()` within the child process and terminate, leaving the parent shell process in its original directory. Only a built-in executed directly inside the parent shell's process context can invoke the `sys_chdir()` system call on the running shell process.
2. **How many directories are in your PATH, and in which one is ls?**
   - The `$PATH` variable contains **12** directory paths (verified via `echo $PATH | tr ':' '\n' | wc -l`).
   - The `ls` executable is located in `/usr/bin/ls` (and resolved via `/usr/bin`).
3. **What exit code does a command that is not found return?**
   - A command that is not found returns exit code **127**. By POSIX conventions, 127 is reserved to indicate that the shell attempted to locate the executable across all directories in `$PATH` and found no matching file.
4. **Which option of ls sorts by modification time?**
   - The `-t` option (or `--sort=time`) instructs `ls` to sort directory entries by modification timestamp, showing the newest entries first.

Part 2. What the Shell Does to Your Line

1. Test Directory Setup

A sandbox directory `play` was populated with representative test files to study pathname expansion and argument parsing:

```
❱ mkdir -p play && cd play
❱ touch a.c b.c file1.txt file2.txt notes.txt "alfa beta.txt"
❱ ls
a.c  'alfa beta.txt'  b.c  file1.txt  file2.txt  notes.txt
```

Figure 2.1 – Files created in the play directory

2. Prediction vs. Real Output Analysis

The table below records the predictions made prior to execution, the actual output observed on the system, and the technical explanation of the shell expansion mechanism:

Command	My prediction	Real output	Explanation / Difference
`echo *.c`	`a.c b.c`	`a.c b.c`	Pathname expansion matches all `.c` files in directory.
`echo file?.txt`	`file1.txt file2.txt`	`file1.txt file2.txt`	Single-character wildcard `?` matches `1` and `2`.
`echo [a-b]*`	`a.c alfa beta.txt b.c`	`a.c alfa beta.txt b.c`	Character range `[a-b]` matches files starting with 'a' or 'b'.
`echo *.xyz`	`*.xyz`	`*.xyz`	With `nullglob` disabled, unmatched globs remain literal strings.
`ls alfa beta.txt`	error: no such file	`ls: cannot access 'alfa': ...` `ls: cannot access 'beta.txt': ...`	Unquoted space causes word splitting into two distinct arguments.
`ls "alfa beta.txt"`	`'alfa beta.txt'`	`'alfa beta.txt'`	Double quotes preserve whitespace, passing a single argument.
`echo '$HOME' "$HOME" \$HOME`	`$HOME /home/calin $HOME`	`$HOME /home/calin $HOME`	Single quotes prevent expansion; double quotes expand; `\` escapes `$`.
`echo {1..5} ; echo $((7 * 6))`	`1 2 3 4 5` `42`	`1 2 3 4 5` `42`	Brace expansion generates sequence; `$((...))` evaluates arithmetic.
`echo "today is $(date +%A), I am $(whoami)"`	`today is Monday, I am calin`	`today is Monday, I am calin`	Command substitution `$(...)` evaluates nested subcommands.
`touch log{1..3}.txt && ls log*`	`log1.txt log2.txt log3.txt`	`log1.txt log2.txt log3.txt`	Brace expansion creates 3 filenames, then listed by glob `log*`.

3. Globbing & Expansion Verification

```
❱ echo *.c
a.c b.c
❱ echo file?.txt
file1.txt file2.txt
❱ echo [a-b]*
a.c alfa beta.txt b.c
❱ echo *.xyz
*.xyz
```

Figure 2.2 – Globbing results

4. Word Splitting on Whitespace and Escaping

Executing `ls alfa beta.txt` demonstrates word splitting on internal field separators (`$IFS`):

```
❱ ls alfa beta.txt
ls: cannot access 'alfa': No such file or directory
ls: cannot access 'beta.txt': No such file or directory
❱ ls "alfa beta.txt"
'alfa beta.txt'
❱ ls alfa\ beta.txt
'alfa beta.txt'
```

Figure 2.3 – The ls error on a name with a space, and the fix

5. Variable Quoting Semantics

```
❱ echo '$HOME' "$HOME" \$HOME
$HOME /home/calin $HOME
```

Figure 2.4 – Single quotes, double quotes and backslash

Observe (Part 2 Answers)

1. **Prediction vs. Real Output:** The predictions matched the real outputs across all test cases. The mechanics of brace expansion, pathname expansion, parameter expansion, and command substitution strictly followed POSIX ordering.
2. **`echo` knows nothing about files, yet `echo *.c` printed file names. Who produced them?**
   - The shell (Bash) expanded `*.c` via pathname expansion (globbing) **before** `echo` was invoked. The shell inspected the filesystem entries in the current directory matching pattern `*.c`, substituted the tokens `a.c` and `b.c` into the argument list (`argv`), and only then executed `echo` with those expanded arguments.
3. **Why did `ls alfa beta.txt` fail? Give two different ways to fix it.**
   - The command failed because the shell performs word splitting using whitespace (`$IFS`). Because the space character was unquoted and unescaped, the shell split `alfa beta.txt` into two separate arguments: `argv[1] = "alfa"` and `argv[2] = "beta.txt"`. The `ls` binary attempted to find both files independently, neither of which existed.
   - **Fix 1 (Quoting):** `ls "alfa beta.txt"` (or `ls 'alfa beta.txt'`)
   - **Fix 2 (Backslash Escaping):** `ls alfa\ beta.txt`
4. **What is the difference between single and double quotes? Note: ls may show the file with a space as `'alfa beta.txt'` (in quotes). Explain why it does that.**
   - **Single quotes (`'...'`):** Completely preserve the literal value of every enclosed character. Variable expansion (`$VAR`), command substitution (`$(...)`), arithmetic evaluation (`$((...))`), and backslash escaping are disabled.
   - **Double quotes (`"..."`):** Suppress word splitting and globbing, but preserve variable expansion, command substitution, arithmetic expansion, and backslash escape sequences.
   - **Why `ls` displays `'alfa beta.txt'` in quotes:** GNU `ls` defaults to `--quoting-style=shell-escape` when stdout is attached to a terminal. It quotes filenames containing spaces or shell metacharacters so the user can immediately distinguish spaces that belong to a single file and safely copy-paste the output back into the shell.

Part 3. Redirection, Chaining and Jobs

1. Separating Standard Streams (`stdout` vs `stderr`)

Standard streams are separated using file descriptor redirection operators `>` (`1>`) and `2>`:

```
❱ ls /etc /nope > out.txt 2> err.txt
❱ wc -l out.txt ; cat err.txt
231 out.txt
ls: cannot access '/nope': No such file or directory
```

Figure 3.1 – stdout and stderr in separate files

2. Comparing Redirection Evaluation Order

File descriptor redirections are evaluated strictly from left to right:

```
# Order A: Redirect stdout (fd 1) to file, then duplicate fd 1 into stderr (fd 2)
❱ ls /etc /nope > all_a.txt 2>&1
❱ grep nope all_a.txt
ls: cannot access '/nope': No such file or directory

# Order B: Duplicate fd 1 into fd 2 (still terminal), then redirect stdout (fd 1) to file
❱ ls /etc /nope 2>&1 > all_b.txt
ls: cannot access '/nope': No such file or directory
❱ grep nope all_b.txt || echo "nope not in all_b.txt"
nope not in all_b.txt
```

Figure 3.2 – Comparing the order of redirections

In Order B, stderr remained attached to the terminal console because the duplication `2>&1` occurred before `> all_b.txt` redirected stdout to the file.

3. Chaining Operators and Stdin Redirection

```
❱ echo first > notes.tmp ; echo second >> notes.tmp ; cat notes.tmp
first
second
❱ wc -l < access.log
500
❱ ls /nope 2> /dev/null || echo "failed with code $?"
failed with code 2
❱ mkdir -p tmp && cd tmp && echo "I am in $(pwd)" ; cd ..
I am in /home/calin/code/os-labs/lab3/tmp
```

Figure 3.3 – The ; && || operators

4. Background Job Control

Running a process asynchronously with `&`, inspecting active jobs, and terminating via job ID:

```
❱ sleep 100 &
[1] 108420
❱ jobs
[1]+  Running                 sleep 100 &
❱ kill %1
[1]+  Terminated              sleep 100
❱ jobs
```

Figure 3.4 – Running a background job and stopping it

Observe (Part 3 Answers)

1. **Which file got the error message, and which got the listing?**
   - `err.txt` received the error message (`ls: cannot access '/nope': No such file or directory`) via standard error redirection (`2> err.txt`).
   - `out.txt` received the directory listing of `/etc` via standard output redirection (`> out.txt`).

2. **What is the difference between `> all.txt 2>&1` and `2>&1 > all.txt`? Try both and explain why the result differs.**
   - The shell evaluates redirection operators strictly from **left to right**.
   - In `> all.txt 2>&1`: File descriptor 1 (`stdout`) is redirected to point to the open file `all.txt`. Next, `2>&1` duplicates fd 1 into file descriptor 2 (`stderr`), causing fd 2 to point to the exact same open file. Consequently, both standard output and error messages are recorded in `all.txt`.
   - In `2>&1 > all.txt`: `2>&1` duplicates fd 1 into fd 2 first. At this stage, fd 1 is still connected to the terminal screen; hence, fd 2 becomes bound to the terminal screen. Next, `> all.txt` redirects fd 1 to `all.txt`. Therefore, error messages continue printing to the user's terminal, and only stdout is written to `all.txt`.

3. **Why does `wc -l < access.log` print only a number, while `wc -l access.log` also prints the file name?**

- When invoked with `access.log` as a command-line argument, `wc` opens the file by name via `open()` and displays both the count and the filename.
- When using `< access.log`, the shell binds `access.log` to standard input (`fd 0`). `wc` receives zero filename parameters in `argv`, reads anonymously from its stdin stream, and outputs only the counted number.

4. **Explain each operator in one sentence: `;`, `&&`, `||`.**
   - `;` executes the subsequent command sequentially and unconditionally, regardless of the exit code of the preceding command.
   - `&&` executes the subsequent command only if the preceding command succeeded with an exit code of 0 (logical AND).
   - `||` executes the subsequent command only if the preceding command failed with a non-zero exit code (logical OR).

Part 4. Answer Questions with One-Liners

All questions were solved using idiomatic POSIX pipelines combining `grep`, `cut`, `sort`, `uniq`, `wc`, `head`, and `tail`. The commands are organized in `oneliners.sh`.

Q1. How many requests failed?

```
❯ grep -c FAIL access.log
100
```

Figure 4.1 – Number of failed requests

Explanation: `grep -c FAIL` counts all lines containing the `FAIL` outcome flag. Exactly 100 out of 500 requests failed.

Q2. How many different pages were requested, and which?

```
❯ cut -d' ' -f5 access.log | sort -u
/page0
/page1
/page2
/page3
```

Figure 4.2 – Distinct pages requested

Explanation: Field 5 contains the requested endpoint URL. Piping to `sort -u` extracts and deduplicates the endpoints, revealing 4 distinct pages: `/page0`, `/page1`, `/page2`, `/page3`.

Q3. How many requests did each page get?

```
❯ cut -d' ' -f5 access.log | sort | uniq -c
    125 /page0
    125 /page1
    125 /page2
    125 /page3
```

Figure 4.3 – Requests per page

Explanation: Each page received exactly 125 requests. Total validation: $125 \times 4 = 500$, matching the total dataset size.

Q4. Which user or users have the most failed requests, and how many?

```
❯ grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn | head -2
     15 user5
     15 user3
```

Figure 4.4 – Users with most failed requests

Explanation: Filtering `FAIL` records and aggregating on username (field 3) shows that **user3** and **user5** are tied for the highest failure count with **15 failures each**. Breakdown: `user0` (14), `user1` (14), `user2` (14), `user3` (15), `user4` (14), `user5` (15), `user6` (14). Total sum: $14 \times 5 + 15 \times 2 = 70 + 30 = 100$ (matching Q1).

Q5. How many requests did user3 make, and how many of them failed?

```
❯ grep 'user3' access.log | cut -d' ' -f4 | sort | uniq -c
     15 FAIL
     57 OK
```

Figure 4.5 – user3 total requests and failures

Explanation: `user3` initiated a total of **72 requests** ($15 + 57 = 72$), with **15 failures** and 57 successful requests.

Q6. Print the last 3 failed requests, showing only time and user.

```
❯ grep FAIL access.log | tail -3 | cut -d' ' -f2,3
10:10 user0
10:15 user5
10:20 user3
```

Figure 4.6 – Last 3 failed requests

Explanation: `grep FAIL` isolates all failure records, `tail -3` selects the final three occurrences, and `cut -d' ' -f2,3` outputs only the timestamp and username columns.

Q7. Which login shells appear in /etc/passwd, and how many accounts use each?

```
❯ cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
     47 /usr/bin/nologin
      2 /usr/bin/bash
      1 /usr/bin/zsh
      1 /usr/bin/git-shell
      1 /sbin/nologin
      1 /bin/bash
```

Figure 4.7 – Login shells in /etc/passwd

Explanation: Field 7 in `/etc/passwd` (colon delimiter) defines the user's login shell. On this Arch Linux host, non-interactive system accounts use `/usr/bin/nologin` (47 accounts) and `/sbin/nologin` (1), while interactive shells include `/usr/bin/bash` (2), `/bin/bash` (1), `/usr/bin/zsh` (1), and restricted `/usr/bin/git-shell` (1).

Q8. PREDICT, then explain: why do `ls /etc | wc -l` and `ls -l /etc | wc -l` differ by exactly one?

```
❯ ls /etc | wc -l ; ls -l /etc | wc -l
230
231
```

Figure 4.8 – Difference between ls and ls -l

- **Prediction:** `ls -l` outputs exactly one more line than `ls` because `ls -l` prepends an aggregate directory block allocation summary header on line 1 (`total <blocks>`).

- **Explanation:** When stdout is connected to a pipe, plain `ls` outputs one directory entry per line without metadata headers (230 entries). In contrast, `ls -l` prints an initial summary line indicating the total filesystem disk blocks allocated by the directory contents, followed by the 230 detailed entry lines. The difference is precisely that single header line ($231 - 230 = 1$).

Bonus (+1): Regex for Minute Ending in 0 and Request Failed

```
❱ grep -E ' 10:[0-9]0 .* FAIL' access.log | head -5
2026-10-05 10:10 user3 FAIL /page2
2026-10-05 10:20 user6 FAIL /page0
2026-10-05 10:30 user2 FAIL /page2
2026-10-05 10:40 user5 FAIL /page0
2026-10-05 10:50 user1 FAIL /page2
❱ grep -E ' 10:[0-9]0 .* FAIL' access.log | wc -l
50
```

Explanation: The extended regular expression `' 10:[0-9]0 .* FAIL'` matches timestamps where the minute digit ends with `0` (`10:00`, `10:10`, …, `10:50`) followed by a failure status. Exactly **50 requests** satisfy both conditions.

Full Run of `oneliners.sh`

```
❱ bash oneliners.sh
100
/page0
/page1
/page2
/page3
    125 /page0
    125 /page1
    125 /page2
    125 /page3
     15 user5
     15 user3
     15 FAIL
     57 OK
10:10 user0
10:15 user5
10:20 user3
     47 /usr/bin/nologin
      2 /usr/bin/bash
      1 /usr/bin/zsh
      1 /usr/bin/git-shell
      1 /sbin/nologin
      1 /bin/bash
230
231
2026-10-05 10:10 user3 FAIL /page2
...
```

Figure 4.9 – Full run of oneliners.sh

Part 5. Your First Script

1. Script Implementation (`report.sh`)

The script checks for input file existence, reports error diagnostics to `stderr` with a non-zero exit code upon failure, prints aggregate counts, and iterates through `user0` to `user6` to report per-user failure tallies:

```
#!/bin/bash
# usage: ./report.sh access.log
log=$1
if [ ! -f "$log" ]; then
```

```
    echo "no such file: $log" >&2
    exit 1
fi
total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")
echo "$total requests, $fails failed"
# TODO: for each of user0 ... user6, print "userN: X failures"
for i in {0..6}; do
    failures=$(grep "user$i" "$log" | grep -c FAIL)
    echo "user$i: $failures failures"
done
```

Figure 5.1 – Contents of report.sh

2. Execution on Valid Access Log

```
❱ chmod +x report.sh
❱ ./report.sh access.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
```

Figure 5.2 – Running the script on access.log

3. Defensive Error Handling on Missing File

```
❱ ./report.sh nope.log ; echo "exit code: $?"
no such file: nope.log
exit code: 1
```

Figure 5.3 – Error handling for a missing file

If executed without execute permissions (`chmod -x`):

```
❱ chmod -x report.sh
❱ ./report.sh access.log
bash: ./report.sh: Permission denied
❱ chmod +x report.sh
```

Observe (Part 5 Answers)

1. **What does the first line `#!/bin/bash` do, and what happens if you run `./report.sh` before `chmod +x` ?**
   - `#!/bin/bash` is the **shebang** directive. When an executable text script is passed to the kernel's `execve()` system call, the kernel inspects the first two magic bytes (`0x23 0x21`). Finding `#!` , it treats the remainder of the line as the absolute path of the interpreter binary to launch, invoking `/bin/bash` with the script path as its argument.
   - If executed before `chmod +x` , the file lacks execute permission bits in its inode metadata. The `execve()` system call immediately fails with POSIX error code `EACCES` (**Permission denied**), and Bash reports `bash: ./report.sh: Permission denied` .

2. **Why must you type `./report.sh` and not just `report.sh` ?**
   - When a command name does not contain any directory slash (`/`), the shell searches only through the colon-delimited directory paths registered in the `$PATH` environment variable.
   - The current working directory (`.`) is excluded from `$PATH` for fundamental security reasons. If `.` were in `$PATH` , an attacker or untrusted user could plant a malicious executable named `ls` or `cd` in a shared directory (e.g. `/tmp`), tricking another user into executing it. Typing `./report.sh` supplies an explicit relative path containing a slash, bypassing `$PATH` search and directing the shell to execute the script in the current directory.

3. **Why does the script write the error with `>&2` and end with `exit 1`?**
   - `>&2` redirects the error message to file descriptor 2 (standard error, `stderr`). In standard Unix architecture, diagnostic messages must be isolated from data streams on standard output (`stdout`, fd 1). If the caller redirects output to a file (`./report.sh log.txt > report.out`), error messages continue printing to the terminal rather than silently corrupting the output data.
   - `exit 1` terminates the script process with a non-zero exit status. In POSIX conventions, 0 indicates clean success while non-zero (1...255) indicates an error. This allows invoking scripts and automated CI pipelines to catch failures using `if`, `&&`, or `||`.

Bonus (+1): Scheduled Execution via Cron

The script was configured for automated recurring execution by adding an entry to the user's crontab (`crontab -e`):

```
* * * * * /home/calin/code/os-labs/lab3/report.sh /home/calin/code/os-labs/lab3/access.log
>> /home/calin/code/os-labs/lab3/report.log 2>&1
```

After two execution cycles, `report.log` records the expected automated runs:

```
❱ cat report.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
```

Conclusions

This laboratory work explored the foundational architecture, execution pipeline, and text processing utilities of the Linux POSIX shell:

1. **Interactive Navigation and Built-in Mechanics:** Reverse history search (`Ctrl+R`) and programmable tab completion significantly accelerate workflow. Demonstrating why `cd` must be a built-in highlighted the Unix process boundary: child processes created via `fork()` cannot alter the parent shell's working directory table in the kernel.
2. **Deterministic Shell Expansion Sequence:** The shell executes command lines through a rigorous parsing order: brace expansion, parameter/variable expansion, arithmetic expansion, command substitution, word splitting, pathname expansion (globbing), and quote removal. Understanding single versus double quote semantics proved essential for preventing unintended tokenization of filenames containing whitespace.
3. **File Descriptor Redirection and Stream Plumbing:** File descriptor manipulations (`>` , `>>` , `<` , `2>` , `2>&1`) operate strictly left-to-right. Differentiating `> all.txt 2>&1` from `2>&1 > all.txt` illustrated how file table pointers are duplicated. Conditional chaining operators (`;` , `&&` , `||`) and job control primitives (`&` , `jobs` , `kill`) provided fine-grained control over process execution flows.
4. **Composable Unix Filter Pipelines:** Combining atomic utilities (`grep` , `cut` , `sort` , `uniq` , `wc` , `tail`) yielded powerful one-liner solutions for parsing structured server access logs, verifying dataset consistency, and aggregating failure metrics without custom high-level programming.
5. **Robust Shell Script Authoring:** Authoring `report.sh` reinforced core Unix scripting disciplines: shebang line resolution via the kernel loader, relative path safety excluding `.` from `$PATH` , standard stream separation via `>&2` , explicit exit codes, and automated periodic invocation through `cron` .